

FL-04 — Ship an Automation Workflow v2

Assignment Code: FL-04

Track: General AI Fluency

Week: 4

Phase: Build

Workload: 7 hours

1. Pipeline Selection

Selected Pipeline: Draft → Critique → Revise

Real Task:

“Creating LinkedIn posts about completed projects or achievements.”

Why I selected this task:

This task is useful because I regularly need to communicate completed technical projects and achievements professionally. Writing a LinkedIn post manually requires deciding what information to include, organizing it clearly, choosing an appropriate tone, and then reviewing the wording. A Draft → Critique → Revise workflow can separate content generation from quality checking and revision.

Input:

Information about a completed project or achievement, including the project name, purpose, technologies used, main features, challenges, and what was accomplished.

Desired Final Output:

A polished LinkedIn post that accurately describes the project, communicates the main achievement clearly, uses a professional but natural tone, and does not invent unsupported claims or metrics.

2. Workflow Design

Workflow Overview

The workflow consists of three main steps:

Input → Draft → Critique → Revise → Final Output

Step 1 — Draft

Purpose:

Create the first version of a LinkedIn post from the original project information..

Input:

Project information supplied by me, including the project name, purpose, technologies, features, challenges, and achievements.

Action:

The AI converts the supplied information into a concise and engaging LinkedIn post while using only the facts provided.

Output:

A first-draft LinkedIn post.

Handoff to Step 2:

The completed draft is passed to the critique step for review.

Step 2 — Critique**Purpose:**

Identify weaknesses or inaccuracies in the draft before producing the final post.

Input:

The original project information and the draft produced by Step 1.

Action:

The AI reviews the draft for:

- Clarity
- Accuracy
- Structure
- Completeness
- Tone
- Unsupported claims
- Weak or missing information
- Excessive repetition
- Generic or overly promotional language

Output:

A structured critique containing specific problems and recommended improvements.

Handoff to Step 3:

The original input, draft, and critique are passed to the revision step.

Step 3 — Revise

Purpose:

Create the improved final LinkedIn post.

Input:

Original project information + draft + critique.

Action:

The AI applies useful feedback while preserving factual accuracy and the user's original project details.

Output:

A final polished LinkedIn post ready for human review.

3. Workflow Diagram

Original Input

↓
Step 1: Draft

↓
Draft

↓
Step 2: Critique

↓
Critique

↓
Step 3: Revise

↓
Final Output

4. Tool Used

Tool/Platform:

Claude Project

Why I selected this tool:

I selected Claude Project because it allows me to keep the workflow instructions and prompts in one place and repeatedly apply the same process to different project inputs. It was suitable for testing the Draft → Critique → Revise pipeline across five real project examples.

How the workflow is structured:

I structured the workflow into three sequential stages. First, the **Draft** prompt takes the original project information and produces an initial LinkedIn post. The resulting draft is then passed to the **Critique** prompt, which reviews it for accuracy, clarity, structure, completeness, tone, repetition, and unsupported claims. Finally, the original input, draft, and critique are passed to the **Revise** prompt, which applies the useful feedback and produces the final revised LinkedIn

post. I then perform a human review of the final output before considering the workflow complete.

5. Prompts and Configuration

Step 1 — Draft Prompt

You are the Draft stage of a three-step LinkedIn post workflow.

Your task is to create the first draft of a LinkedIn post from the project information provided by the user.

Rules:

1. Use only information explicitly provided by the user.
2. Do not invent technologies, features, results, metrics, clients, users, awards, or business outcomes.
3. If important information is missing, do not guess it.
4. Keep the tone professional, natural, and suitable for LinkedIn.
5. Focus on what was built, what problem it addresses, important technical or design decisions, and what was learned or accomplished.
6. Avoid exaggerated claims and generic AI-generated phrases.
7. Make the post easy to read with short paragraphs.
8. Do not perform the critique or final revision yet.

Return only the first-draft LinkedIn post.

USER PROJECT INPUT:

[PASTE PROJECT INFORMATION HERE]

Step 2 — Critique Prompt

You are the Critique stage of a three-step LinkedIn post workflow.

Review the LinkedIn draft against the original project information.

Evaluate it for:

1. Accuracy
2. Clarity
3. Structure
4. Completeness
5. Professional but natural tone

6. Unsupported or exaggerated claims
7. Missing important project information
8. Repetition or unnecessary wording
9. Generic or overly promotional language
10. Whether the post accurately represents the user's actual work

Do not rewrite the complete post.

Return a structured critique with:

- What works well
- Accuracy issues
- Missing information
- Tone/style issues
- Specific recommended changes

If the draft is factually accurate, explicitly state that no factual issue was identified.

ORIGINAL PROJECT INPUT:

[PASTE ORIGINAL INPUT HERE]

DRAFT:

[PASTE STEP 1 DRAFT HERE]

Step 3 — Revise Prompt

You are the Revision stage of a three-step LinkedIn post workflow.

Create the final LinkedIn post using:

1. The original project information
2. The first draft
3. The critique

Rules:

1. Preserve factual accuracy.
2. Never add information that is not supported by the original project input.
3. Apply useful feedback from the critique.
4. Remove unsupported claims, exaggeration, repetition, and generic wording.
5. Keep the tone professional, confident, and natural.
6. Make the post concise and easy to read.
7. Do not claim measurable results unless those results were provided in the original input.
8. Do not mention that an AI workflow was used to create the post.
9. Return only the final revised LinkedIn post.

ORIGINAL PROJECT INPUT:
[PASTE ORIGINAL INPUT HERE]

DRAFT:
[PASTE STEP 1 DRAFT HERE]

CRITIQUE:
[PASTE STEP 2 CRITIQUE HERE]

6. Workflow Run 1

Input:
BMS project information.

Step 1 — Draft Output:

Excited to share a project I've been working on: a hybrid Business Inventory Management System designed to help businesses manage products, stock, and inventory operations digitally.

The system includes:

- Product and inventory management
- Barcode-based inventory tracking
- User authentication with JWT-based authorization
- A dashboard for viewing inventory information
- A responsive web interface
- A backend API connecting the frontend to the database

On the backend, I used ASP.NET Core Web API with C#, connected to a PostgreSQL database hosted on Supabase via Npgsql. The frontend was built with React and Vite, using Axios to communicate with the API. Docker was used as part of the development setup.

My role covered the backend API, database integration, authentication, and frontend integration.

One of the main challenges was reliably connecting the ASP.NET Core backend to the hosted PostgreSQL database, working through configuration and connection issues during development.

Through this project, I gained hands-on experience with full-stack development, building REST APIs, integrating databases, implementing authentication, and connecting a React frontend to an ASP.NET Core backend.

Always glad to discuss the technical details or exchange ideas with others working on similar systems.

Step 2 — Critique Output:

What works well

- Accurately lists all features and technologies exactly as provided — no invented items detected.
- Clear structure: intro → features → tech stack → role → challenge → learning.
- Tone is professional and mostly free of hype language.
- Challenge and learning sections are honest and specific rather than generic.

Accuracy issues

No factual issue was identified. All technologies, features, and claims trace directly back to the user's input. Nothing appears invented (no fabricated metrics, users, results, or awards).

Missing information

- The word "hybrid" (from "hybrid inventory management system") is used but never explained — the original input doesn't clarify what "hybrid" refers to (e.g., hybrid of web/desktop, or hybrid architecture). This could confuse readers since it's an unexplained technical term.
- No mention of project status (e.g., ongoing, completed, personal/academic/professional context) — this is missing from the source input too, but its absence may leave readers unsure what stage the project is at.

Tone/style issues

- The closing line ("Always glad to discuss the technical details or exchange ideas with others working on similar systems") borders on a generic LinkedIn closer. It's not egregious, but it's a fairly common stock phrase and could feel slightly templated rather than personal.
- The phrase "Excited to share" is a common LinkedIn opener — mildly generic, though not exaggerated or false.
- Docker is mentioned only in passing ("used as part of the development setup") without specifying its role (e.g., containerizing the backend, local dev environment) — this vagueness is minor but slightly weakens the technical credibility section.

Repetition or unnecessary wording

- "Backend API" is referenced three times (features list, tech stack paragraph, and role paragraph). Not incorrect, but slightly repetitive across sections.
- The features list and the "my role" paragraph overlap somewhat (authentication, backend API, frontend integration are mentioned in both), which could be tightened.

Specific recommended changes

1. Clarify or remove "hybrid" if no further detail is available — as written, it introduces an undefined term.

2. Consider trimming or replacing the generic opener ("Excited to share") and generic closer with more specific, less templated phrasing.
3. Reduce redundancy between the features list and the "role" paragraph — e.g., the role paragraph could focus on *what the user personally did* rather than re-listing feature names.
4. Optionally specify Docker's actual function in the project (if the user can confirm), since currently it reads as a vague technology-list inclusion rather than a described contribution.
5. Keep all technical claims as-is — do not add metrics, outcomes, or scale indicators, since none were provided.

Step 3 — Final Revised Output:

I built a Business Inventory Management System to help businesses manage products, stock, and inventory operations digitally.

The system includes:

- Product and inventory management
- Barcode-based inventory tracking
- User authentication with JWT-based authorization
- A dashboard for viewing inventory information
- A responsive web interface

On the backend, I used ASP.NET Core Web API with C#, connected to a PostgreSQL database hosted on Supabase via Npgsql. The frontend was built with React and Vite, using Axios to communicate with the API, with Docker as part of the development setup.

My focus was on the backend API, database integration, authentication, and connecting the frontend to the backend.

One of the main challenges was reliably connecting the ASP.NET Core backend to the hosted PostgreSQL database, working through configuration and connection issues during development.

Through this project, I gained hands-on experience with full-stack development, building REST APIs, integrating databases, implementing authentication, and connecting a React frontend to an ASP.NET Core backend.

You are the Revision stage of a three-step LinkedIn post workflow.
Create the final LinkedIn post using:

1. The original project information
2. The first draft
3. The critique

Rules:

1. Preserve factual accuracy.
2. Never add information that is not supported by the original project input.
3. Apply useful feedback from the critique.

Show more

I built a Business Inventory Management System to help businesses manage products, stock, and inventory operations digitally.

The system includes:

- Product and inventory management
- Barcode-based inventory tracking
- User authentication with JWT-based authorization
- A dashboard for viewing inventory information
- A responsive web interface

On the backend, I used ASP.NET Core Web API with C#, connected to a PostgreSQL database hosted on Supabase via Npgsql. The frontend was built with React and Vite, using Axios to communicate with the API, with Docker as part of the development setup.

My focus was on the backend API, database integration, authentication, and connecting the frontend to the backend.

One of the main challenges was reliably connecting the ASP.NET Core backend to the hosted PostgreSQL database, working through configuration and connection issues during development.

Human Check:

I checked the final post against the original BMS project information to make sure the technologies, features, and responsibilities were accurate. I verified the use of ASP.NET Core, C#, PostgreSQL, Supabase, Npgsql, React, Vite, Axios, and Docker. I also checked that the post did not introduce unsupported metrics, business results, client information, or other claims that were not included in the original input. I accepted the revised version after confirming that the generic opening and closing from the draft had been removed and that the main technical information remained accurate.

7. Workflow Run 2

Input:

Project: Museum of Digital Craft

Project purpose:

I worked on a premium portfolio website concept designed as an interactive digital museum rather than a conventional portfolio website.

Main features:

- Grand Hall homepage
- Museum-wide navigation
- Entrance experience
- Custom cursor
- Smooth scrolling
- Ambient visual atmosphere
- Room-to-room transitions
- Gallery and exhibit system
- Artwork frame component
- Dynamic gallery and exhibit routing
- Automatic content discovery
- MDX-based case study rendering
- Archive section
- Individual self-contained folders for exhibits

Technologies and tools used:

- Next.js
- React
- TypeScript
- Tailwind CSS
- MDX
- GSAP
- Motion
- Lenis
- Zod
- React Hook Form

What I completed:

I developed the website structure and interactive museum experience, including the Grand Hall, navigation, entrance sequence, gallery system, exhibit routing, case study rendering, and archive experience.

Design direction:

The goal was to create a premium, modern, editorial-style digital experience rather than a basic portfolio grid.

Challenge:

A major challenge was organizing a large number of components and features into a coherent, maintainable project structure while preserving the intended visual and interactive experience.

Learning:

I gained experience with advanced frontend architecture, animation, interactive navigation, content-driven pages, and structuring a large Next.js project.

Important instruction:

Do not claim that the website won an award, attracted clients, generated revenue, or received a specific number of visitors unless those results are actually known.

Draft Output:

I built the Museum of Digital Craft, a portfolio concept reimaged as an interactive digital museum instead of a conventional portfolio website.

The experience is structured around a Grand Hall homepage, with museum-wide navigation guiding visitors through an entrance sequence and room-to-room transitions. A custom cursor, smooth scrolling, and an ambient visual atmosphere reinforce the sense of moving through a physical space rather than scrolling a webpage.

At the core is a gallery and exhibit system, built with an artwork frame component and dynamic routing for galleries and exhibits. Content discovery is automatic, and case studies are rendered using MDX, with each exhibit organized as its own self-contained folder. An archive section rounds out the experience.

On the technical side, I built this with Next.js, React, and TypeScript, styled with Tailwind CSS. Animation and interaction were handled with GSAP, Motion, and Lenis, while Zod and React Hook Form supported form handling and validation.

I developed the full website structure and interactive experience myself — from the Grand Hall and navigation to the gallery system, exhibit routing, case study rendering, and archive.

The design direction was intentionally premium and editorial, moving away from a basic portfolio grid toward something closer to a curated exhibition.

One of the bigger challenges was organizing a large number of components and features into a coherent, maintainable structure while preserving the intended visual and interactive feel.

This project pushed my understanding of advanced frontend architecture, animation, interactive navigation, content-driven page structures, and working with a large-scale Next.js codebase.

Critique Output:**What works well**

- Covers all listed features and technologies without introducing anything absent from the source input.
- Logical flow: concept → experience/navigation → gallery/exhibit system → tech stack → role → design direction → challenge → learning.
- Conveys the "digital museum" concept clearly and distinctively, which matches the project's stated purpose.
- Challenge and learning sections stay grounded in what was actually provided.

Accuracy issues

No factual issue was identified. All features, technologies, and descriptions map directly to the original input. No awards, clients, revenue, or visitor numbers are claimed, in line with the explicit instruction.

Missing information

- Nothing substantive is missing — all provided fields (purpose, features, tech, completed work, design direction, challenge, learning) are represented.
- One minor omission: the input frames this as a "premium portfolio website concept," implying it may be conceptual/demo work rather than a live client project. The draft doesn't clarify whether this is a personal concept project — this ambiguity could let readers assume it's a commissioned or live production site. Worth clarifying if the user can confirm, though the draft doesn't explicitly claim otherwise either.

Tone/style issues

- Some phrasing leans slightly promotional/evocative rather than plainly descriptive, e.g., "reinforce the sense of moving through a physical space," "something closer to a curated exhibition," and "pushed my understanding of." These aren't false, but they're more editorial/flourished than strictly necessary — bordering on stylized marketing language rather than a straightforward technical account.
- "This project pushed my understanding of..." is a slightly generic closing construction common in portfolio posts; could be more specific/personal.
- The post is long relative to typical LinkedIn scroll behavior — six paragraphs of fairly dense description. Not incorrect, but could be tightened for readability.

Repetition or unnecessary wording

- The gallery/exhibit system is described in the features paragraph and largely re-described in the "role" paragraph ("gallery system, exhibit routing, case study rendering, and archive") — near-duplicate phrasing across two paragraphs.
- "Interactive" is used multiple times (interactive digital museum, interactive navigation, interactive experience) — minor repetition that could be varied or trimmed.

Specific recommended changes

1. Tighten the overlap between the features paragraph and the "role" paragraph — the role paragraph should focus on ownership/scope rather than re-listing the same feature names.
2. Consider trimming some of the more descriptive/editorial phrasing (e.g., "moving through a physical space," "curated exhibition") to keep tone confident and factual rather than promotional.
3. Vary or reduce repeated use of "interactive."
4. Consider shortening overall length by merging the "design direction" note into the opening paragraph, since it directly supports the initial concept description.
5. Keep all technical and feature claims exactly as-is — no factual changes needed.

Final Revised Output:

I built the Museum of Digital Craft, a portfolio concept designed as an interactive digital museum rather than a conventional portfolio website. The goal was a premium, editorial-style experience — closer to a curated exhibition than a basic portfolio grid.

The experience is structured around a Grand Hall homepage, with museum-wide navigation, an entrance sequence, and room-to-room transitions. A custom cursor, smooth scrolling, and an ambient visual atmosphere support the overall feel.

At the core is a gallery and exhibit system with an artwork frame component and dynamic gallery/exhibit routing. Content discovery is automatic, case studies render through MDX, and each exhibit is organized as its own self-contained folder, alongside an archive section.

I built the full website structure and interactive experience — from the Grand Hall and navigation to the gallery system, exhibit routing, case study rendering, and archive.

Technically, this was built with Next.js, React, and TypeScript, styled with Tailwind CSS. GSAP, Motion, and Lenis handled animation and interaction, while Zod and React Hook Form supported form handling and validation.

A major challenge was organizing a large number of components and features into a coherent, maintainable structure while preserving the intended visual and interactive experience.

This project deepened my experience with advanced frontend architecture, animation, interactive navigation, content-driven page structures, and working within a large-scale Next.js codebase.

You are the Revision stage of a three-step LinkedIn post workflow.
Create the final LinkedIn post using:

1. The original project information
2. The first draft
3. The critique

Rules:

1. Preserve factual accuracy.
2. Never add information that is not supported by the original project input.
3. Apply useful feedback from the critique.

Show more

I built the Museum of Digital Craft, a portfolio concept designed as an interactive digital museum rather than a conventional portfolio website. The goal was a premium, editorial-style experience — closer to a curated exhibition than a basic portfolio grid.

The experience is structured around a Grand Hall homepage, with museum-wide navigation, an entrance sequence, and room-to-room transitions. A custom cursor, smooth scrolling, and an ambient visual atmosphere support the overall feel.

At the core is a gallery and exhibit system with an artwork frame component and dynamic gallery/exhibit routing. Content discovery is automatic, case studies render through MDX, and each exhibit is organized as its own self-contained folder, alongside an archive section.

I built the full website structure and interactive experience — from the Grand Hall and navigation to the gallery system, exhibit routing, case study rendering, and archive.

Technically, this was built with Next.js, React, and TypeScript, styled with Tailwind CSS. GSAP, Motion, and Lenis handled animation and interaction, while Zod and React Hook Form supported form handling and validation.

A major challenge was organizing a large number of components and features into a coherent, maintainable structure while preserving the intended visual and interactive experience.

Human Check:

I checked the final post against the original Museum of Digital Craft project information to verify the features, architecture, and technologies mentioned. I specifically checked the Grand Hall, museum navigation, gallery and exhibit system, MDX case studies, archive, and self-contained exhibit structure. I also verified the Next.js, React, TypeScript, Tailwind CSS, GSAP, Motion, Lenis, Zod, and React Hook Form references. I checked that the post did not claim awards, clients, revenue, visitor numbers, or other results that were not provided. I also reviewed the wording to make sure the project was presented as a portfolio concept rather than implying an unverified commercial project.

8. Workflow Run 3

Input:

Project: AI-Assisted Software Requirements and SRS Generation System

Project purpose:

I worked on a web application designed to help users collect, organize, analyze, and convert software requirements into a structured Software Requirements Specification.

Main features:

- User registration and login
- Interactive requirement interview
- Requirement conflict detection
- Requirement categorization
- SRS generation
- IEEE-style SRS structure
- Chatbot-based interaction
- Backend API
- SQLite database

Technologies used:

- Node.js
- Express
- JavaScript
- SQLite
- Hugging Face inference
- HTML/CSS
- PlantUML for system diagrams

What I completed:

I worked on the application structure and features for collecting requirements through an interactive process and generating a structured SRS document.

AI component:

The application uses AI inference to support the requirement-gathering and SRS generation process.

Challenge:

One challenge was designing the workflow so that user requirements could be collected interactively and then transformed into organized software requirements without losing important information.

Learning:

I gained experience with backend development, database integration, AI inference, requirements engineering, and designing workflows that combine traditional software engineering with AI assistance.

Important instruction:

Do not claim that the generated SRS documents are automatically perfect or fully compliant with every IEEE requirement. Human review is still required.

Draft Output:

I worked on a web application designed to help users collect, organize, analyze, and convert software requirements into a structured Software Requirements Specification.

The core idea was to make requirements gathering more interactive and structured. Users go through an interactive requirement interview via a chatbot-based interface, and the system detects conflicts between requirements and categorizes them.

From there, the application generates an SRS document following an IEEE-style structure. AI inference (via Hugging Face) supports both the requirement-gathering process and the SRS generation itself, though the output is meant to support the process, not replace human review — the generated documents still need to be reviewed for accuracy and completeness.

Other features include user registration and login, a backend API, and an SQLite database for storing requirement data. PlantUML is used for generating system diagrams.

On the technical side, I built this with Node.js and Express on the backend, JavaScript throughout, SQLite for the database, and HTML/CSS on the frontend.

I worked on the application structure and the features for collecting requirements interactively and generating the SRS document.

One of the main challenges was designing the workflow so that requirements collected interactively could be transformed into organized, structured output without losing important information along the way.

This project gave me experience with backend development, database integration, AI inference, requirements engineering, and designing workflows that combine traditional software engineering with AI assistance.

Critique Output:

What works well

- All features and technologies from the input are represented accurately, with nothing invented.

- Correctly follows the important instruction: explicitly states the SRS output is not automatically perfect and still requires human review, rather than overclaiming AI capability.
- Logical flow: purpose → interactive process → conflict detection/categorization → SRS generation → AI role → other features → tech stack → role → challenge → learning.
- The disclaimer about human review is handled naturally, not as an awkward disclaimer bolted on.

Accuracy issues

No factual issue was identified. All claims trace back to the original input, and the required caveat about SRS documents not being automatically perfect/fully IEEE-compliant is properly included.

Missing information

- Nothing substantive from the input is missing — all features, technologies, role, challenge, and learning are covered.
- The input doesn't specify whether this is a solo, academic, or team project — the draft doesn't claim either way, which is appropriate given the ambiguity, but worth flagging as an area the user could clarify if desired.

Tone/style issues

- "The core idea was to make requirements gathering more interactive and structured" is a slightly generic transitional sentence — it restates the purpose already given in the opening line rather than adding new information.
- "though the output is meant to support the process, not replace human review" is good content but phrased a bit stiffly/parenthetically; it could be integrated more smoothly.
- Overall tone is appropriately professional and not overly promotional — no exaggerated claims like "revolutionary" or "cutting-edge" are present.

Repetition or unnecessary wording

- The opening sentence and the second paragraph's first sentence both restate the same purpose ("collect, organize... convert requirements" vs. "make requirements gathering more interactive and structured") — redundant framing.
- "I worked on the application structure and features for collecting requirements interactively and generating the SRS document" (role paragraph) closely repeats the opening sentence and the earlier description of features — this is the most noticeable repetition in the draft.
- "structured" and "interactive" are each used multiple times across paragraphs, which is minor but adds to the sense of repetition.

Specific recommended changes

1. Remove or merge the redundant second-paragraph opener with the first paragraph, since they cover the same ground.
2. Tighten the "role" paragraph so it doesn't re-list the same features already described earlier — focus it on ownership/scope instead.
3. Smooth the human-review caveat into a cleaner standalone sentence rather than a mid-sentence qualifier.
4. Reduce repeated use of "interactive" and "structured" by varying word choice where possible.
5. Keep all technical details, AI-role description, and the human-review caveat exactly as-is — these are accurate and appropriately cautious per the instructions.

Final Revised Output:

I worked on a web application designed to help users collect, organize, analyze, and convert software requirements into a structured Software Requirements Specification.

Users go through an interactive requirement interview via a chatbot-based interface. The system then detects conflicts between requirements and categorizes them before generating an SRS document following an IEEE-style structure.

AI inference, via Hugging Face, supports both the requirement-gathering process and the SRS generation itself. The generated documents are meant to support the process, not replace it — human review is still required.

Other features include user registration and login, a backend API, an SQLite database for storing requirement data, and PlantUML for generating system diagrams.

On the technical side, I built this with Node.js and Express on the backend, JavaScript throughout, SQLite for the database, and HTML/CSS on the frontend.

One of the main challenges was designing the workflow so that requirements collected interactively could be transformed into organized, structured output without losing important information along the way.

This project gave me experience with backend development, database integration, AI inference, requirements engineering, and designing workflows that combine traditional software engineering with AI assistance.

Human Check:

I checked the final post against the original project information to verify the application's features, technologies, and AI functionality. I confirmed that the interactive requirement interview, conflict detection, categorization, SRS generation, chatbot interaction, Node.js, Express, SQLite, Hugging Face inference, HTML/CSS, and PlantUML were represented accurately. I also checked the important limitation that the generated SRS documents still require human review and that the post did not claim automatic or complete IEEE compliance. Finally, I

reviewed the revised wording to make sure repeated descriptions of the project were removed without removing important technical information.

9. Workflow Run 4

Input:

Project: Tailor Management System

Project purpose:

I worked on a desktop-based tailor management application designed to help manage customers, orders, designs, employees, and related business information.

Main features:

- Customer management
- Order management
- Design catalog
- Employee management
- Login and registration
- Forgot password functionality
- Dashboard
- CRUD operations for major business entities

Technologies used:

- C#
- C# WinForms
- SQLite
- Object-Oriented Programming
- Database management

Project development:

The application was originally associated with an MS Access database and was converted to use SQLite.

What I completed:

I worked on converting and structuring the application around an object-oriented design while using SQLite as the database backend.

Challenge:

One challenge was moving the application's database functionality from MS Access to SQLite while keeping the application's existing management features functional.

Learning:

I strengthened my understanding of object-oriented programming, desktop application development, database CRUD operations, and connecting a C# application to a relational database.

Important instruction:

Do not claim that the system was deployed to a real tailoring business or used by a specific number of customers unless that actually happened.

Draft Output:

I worked on a Tailor Management System, a desktop application designed to help manage customers, orders, designs, employees, and related business information.

The application covers core business operations through customer management, order management, a design catalog, and employee management, along with login/registration, forgot password functionality, and a dashboard. CRUD operations are implemented across the major business entities.

One notable part of this project was the database migration. The application was originally built around an MS Access database, and I worked on converting it to use SQLite instead, restructuring the application around an object-oriented design in the process.

This meant reworking how the application interacted with its data layer while making sure the existing management features continued to work correctly.

Built with C# and WinForms, the project applies object-oriented programming principles throughout, along with general database management practices.

One of the main challenges was migrating the database functionality from MS Access to SQLite without breaking the application's existing features.

Through this project, I strengthened my understanding of object-oriented programming, desktop application development, database CRUD operations, and connecting a C# application to a relational database.

Critique Output:

What works well

- All features and technologies map directly to the original input; nothing invented.
- Correctly avoids claiming real-world deployment, a specific business client, or customer numbers, in line with the explicit instruction.
- Structure is logical: purpose → features → migration story → tech stack → challenge → learning.

- The MS Access → SQLite migration is framed as the central narrative, which matches what the user emphasized as their main contribution.

Accuracy issues

No factual issue was identified. All claims — features, technologies, the migration work, and the challenge/learning content — are directly supported by the original input.

Missing information

- Nothing substantive is missing; all provided fields (features, tech, development history, completed work, challenge, learning) are represented.
- The input doesn't specify project context (academic, personal, freelance) — the draft doesn't claim any context, which is appropriate given the ambiguity.

Tone/style issues

- Overall tone is professional and appropriately restrained — no exaggerated claims like "revolutionized" or "streamlined operations" are present.
- "One notable part of this project was the database migration" is a slightly generic transitional phrase that could be tightened or removed since the next sentence already conveys this.
- The tone is natural and reads as a genuine technical account rather than promotional copy.

Repetition or unnecessary wording

- The migration from MS Access to SQLite is described three times: in the "development" paragraph, again in the paragraph immediately after ("This meant reworking..."), and a third time in the "challenge" paragraph. This is the most notable redundancy in the draft.
- "Object-oriented design/programming" is mentioned twice (development paragraph and tech stack paragraph) — minor repetition, though not unreasonable given it's a core theme.
- The "This meant reworking how the application interacted with its data layer while making sure the existing management features continued to work correctly" paragraph largely restates the same idea as the surrounding paragraphs and could be merged or cut.

Specific recommended changes

1. Consolidate the three separate mentions of the MS Access → SQLite migration into one clear, non-repetitive explanation, then let the "challenge" paragraph add new detail rather than restate it.
2. Cut or merge the transitional sentence "One notable part of this project was the database migration" since it doesn't add new information.
3. Remove or shorten the standalone paragraph "This meant reworking how the application interacted with its data layer..." since it duplicates the migration/challenge content.

4. Keep the tech stack, feature list, and learning sections as-is — they're accurate and clearly written.
5. Consider shortening the post slightly overall now that repetition is removed, to improve scannability on LinkedIn.

Final Revised Output:

I worked on a Tailor Management System, a desktop application designed to help manage customers, orders, designs, employees, and related business information.

The application covers core business operations through customer management, order management, a design catalog, and employee management, along with login/registration, forgot password functionality, and a dashboard. CRUD operations are implemented across the major business entities.

The application was originally built around an MS Access database. I worked on converting it to use SQLite instead, restructuring the application around an object-oriented design in the process while keeping the existing management features functional.

Built with C# and WinForms, the project applies object-oriented programming principles throughout, along with general database management practices.

One of the main challenges was migrating the database functionality from MS Access to SQLite without breaking any of the application's existing features.

Through this project, I strengthened my understanding of object-oriented programming, desktop application development, database CRUD operations, and connecting a C# application to a relational database.

Human Check:

I checked that the final post accurately represented the Tailor Management System and did not introduce any unsupported claims. I specifically verified the MS Access to SQLite migration, the C# and WinForms technologies, the listed management features, and the object-oriented programming work. I also checked that the final wording did not claim that the system had been deployed to a real tailoring business or used by a specific number of customers.

10. Workflow Run 5

Input:

Project: SyncWorks

Project purpose:

I developed the concept and brand direction for SyncWorks, an AI automation and software company focused on helping businesses automate repetitive processes.

Main focus areas:

- AI automation
- AI agents
- n8n workflows
- Custom software development
- Business process automation

Design and brand direction:

The brand was designed around a premium, modern, enterprise-oriented SaaS aesthetic. The visual direction emphasizes simplicity, professional typography, strong layouts, and a technology-focused identity rather than generic AI imagery.

Automation work:

One of the automation concepts involved connecting business order information from PostEx and WooCommerce and organizing the resulting information into Google Sheets.

Example workflow information:

- PostEx provides shipment/order information.
- The order reference can be used to retrieve additional product information from WooCommerce.
- The resulting information can then be organized into structured business records.

Business data considered:

- Order reference
- Pickup date
- Delivery city
- Status
- COD amount
- Shipping charges
- GST
- Withholding taxes
- Net amount
- Cost
- Profit
- Payment method

Technologies/tools explored:

- n8n
- WooCommerce REST API
- PostEx data
- Google Sheets
- AI automation

- Custom software

Challenge:

A challenge was designing an automation workflow where information from different business systems could be combined into a consistent format for reporting and analysis.

Learning:

I gained experience thinking about business processes as automation workflows and understanding how APIs, workflow automation, and structured business data can work together.

Important instruction:

Do not claim that SyncWorks has a specific number of clients, revenue, employees, or deployed automations unless those facts are actually true.

Draft Output:

I've been working on SyncWorks, a concept and brand direction for an AI automation and software company focused on helping businesses automate repetitive processes.

The core focus areas are AI automation, AI agents, n8n workflows, custom software development, and business process automation.

On the brand side, the direction is a premium, modern, enterprise-oriented SaaS aesthetic — built around simplicity, professional typography, and strong layouts, with a technology-focused identity rather than generic AI imagery.

One of the automation concepts I worked through involved connecting order information between PostEx and WooCommerce. PostEx provides shipment and order data, and the order reference can be used to pull additional product information from WooCommerce. That combined information is then organized into structured business records in Google Sheets.

The business data involved includes order reference, pickup date, delivery city, status, COD amount, shipping charges, GST, withholding taxes, net amount, cost, profit, and payment method.

This concept was explored using n8n, the WooCommerce REST API, PostEx data, and Google Sheets.

A key challenge was designing the workflow so that information coming from different business systems could be combined into a consistent format suitable for reporting and analysis.

Working through this gave me a better understanding of how to think about business processes as automation workflows, and how APIs, workflow automation, and structured business data can work together.

Critique Output:

What works well

- Clearly frames SyncWorks as a concept/brand direction rather than an established, operating company — this matches the input and avoids implying real business traction.
- All focus areas, brand direction details, workflow logic, data fields, and technologies are accurately drawn from the input with nothing invented.
- Correctly avoids any mention of client count, revenue, employees, or deployed automations, per the explicit instruction.
- Logical structure: concept intro → focus areas → brand direction → automation example → data fields → tools → challenge → learning.
- The automation workflow explanation (PostEx → WooCommerce → Google Sheets) is described clearly and accurately reflects the described logic without overstating it as a finished product.

Accuracy issues

No factual issue was identified. The draft consistently frames this as a concept/exploration rather than a deployed system, which matches the original input's framing ("project purpose," "explored," "concept").

Missing information

- Nothing substantive is missing — all input fields (purpose, focus areas, brand direction, automation example, data fields, tools, challenge, learning) are represented.
- The input doesn't clarify whether SyncWorks is a personal venture, a portfolio/concept piece, or an active startup — the draft appropriately doesn't guess this, though the phrase "I've been working on SyncWorks" could unintentionally read as an ongoing active business rather than a conceptual/brand project. Worth flagging for clarity.

Tone/style issues

- "I've been working on SyncWorks, a concept and brand direction for an AI automation and software company" is good, but the opening could be slightly ambiguous about whether SyncWorks is a real operating company or a conceptual project — since the input frames it as "the concept and brand direction," this framing should stay explicit throughout, not just in the opener.
- The listing of business data fields ("order reference, pickup date, delivery city, status, COD amount...") is accurate but reads as a dense, comma-heavy sentence — slightly harder to scan on LinkedIn compared to the rest of the post.
- No exaggerated or promotional language is present (no "revolutionary," "game-changing," etc.) — tone is appropriately grounded.

Repetition or unnecessary wording

- "Automation" and "workflow" are used frequently throughout (automation concepts, automation workflow, workflow automation, business process automation) — expected given the subject matter, but a few instances could be varied for smoother reading.

- The tools paragraph ("This concept was explored using n8n, the WooCommerce REST API, PostEx data, and Google Sheets") largely restates technologies already implied in the automation-example paragraph — minor redundancy, though not a major issue since it consolidates the tech stack in one place.

Specific recommended changes

1. Consider rephrasing the business-data-fields sentence into a short list or two shorter sentences to improve scannability, rather than one long comma-separated run.
2. Reinforce early and consistently that SyncWorks is a concept/brand direction rather than an operating business, to avoid any reader assumption of real deployment or clients — this is mostly handled well already but could be tightened.
3. Vary word choice slightly around "automation/workflow" repetition where natural.
4. Keep the challenge, learning, and technology sections as-is — accurate and clearly written.
5. No factual corrections needed; changes here are purely stylistic/clarity-focused.

Final Revised Output:

I've been developing the concept and brand direction for SyncWorks, an AI automation and software company focused on helping businesses automate repetitive processes.

The core focus areas are AI automation, AI agents, n8n workflows, custom software development, and business process automation.

The brand direction is built around a premium, modern, enterprise-oriented SaaS aesthetic — simplicity, professional typography, and strong layouts, with a technology-focused identity rather than generic AI imagery.

One automation concept I worked through involved connecting order information between PostEx and WooCommerce. PostEx provides shipment and order data, and the order reference can be used to pull additional product information from WooCommerce. That combined information is then organized into structured business records in Google Sheets.

The business data involved includes:

- Order reference, pickup date, delivery city, status
- COD amount, shipping charges, GST, withholding taxes
- Net amount, cost, profit, payment method

This concept was explored using n8n, the WooCommerce REST API, PostEx data, and Google Sheets.

A key challenge was designing the workflow so that information from different business systems could be combined into a consistent format suitable for reporting and analysis.

Working through this gave me a better understanding of how to think about business processes as automation workflows, and how APIs, workflow automation, and structured business data can work together.

Human Check:

I checked that the final post correctly described SyncWorks as a concept and brand direction rather than claiming it was an established company with clients or revenue. I verified the PostEx, WooCommerce, n8n, and Google Sheets workflow details and checked that the listed business data fields were accurate. I also reviewed the final wording to make sure it did not exaggerate the automation work or claim that the workflow had been commercially deployed.

11. Time Comparison

Manual Process

Estimated time for one task manually:

15-20 minutes

Five tasks manually:

75-100

minutes

Workflow Process

Initial setup time:

10-15 minutes

Time spent running five workflow inputs:

20-25 minutes

Time spent checking/editing five outputs:

15-20 minutes

Total workflow time including setup:

45-60 minutes

Time Saved

Manual total: 85 minutes

Workflow total: 52 minutes

Estimated time saved: 33 minutes

Observation:

The workflow likely saves time primarily on repeated runs, not on the first one because the setup cost (learning/writing the prompt structure) eats into savings for a single task and is only "paid off" once you're running multiple posts through it. The main efficiency gain comes from not having to mentally hold all the critique criteria (accuracy, repetition, tone, etc.) yourself each time and the critique stage does that consistently. However, the workflow doesn't eliminate review time entirely: each output still needs a human check for accuracy and voice, so the time saved is more about *consistency and reduced cognitive load* than a dramatic reduction in total minutes, especially at low volume (five tasks). The time saving case gets stronger the more posts you run through it, since setup cost is fixed but per-task time stays roughly flat.

12. What Broke or Needed Improvement

During the five runs, the workflow had the following problems:

1. The draft sometimes repeated the same project information, particularly when explaining the main challenge or technical work.
2. Some drafts contained generic transitional wording that did not add much information and could be removed to make the LinkedIn post more concise.
3. The workflow could not independently determine whether project claims were true beyond the information supplied in the input, so factual accuracy still required human verification.

How I handled these problems:

I used the Critique stage to identify repetition, unnecessary wording, and potential ambiguity before revision. The Revise stage then consolidated repeated information and shortened sections where appropriate. I also kept the original project input available during revision so that unsupported claims could be avoided. Finally, I performed a human check on each final output to verify project details, technologies, and claims before considering the post complete.

13. Human Review

The workflow still requires human checking for:

1. Factual accuracy: verifying that the technologies, features, project status, and achievements mentioned in the post are correct.
2. Personal voice and tone: checking that the final post sounds natural and represents how I would actually describe my work.
3. Claims and context : making sure the AI has not exaggerated the project's results, deployment status, business impact, or level of completion.

Why human review is necessary:

The AI can improve the structure and wording of a LinkedIn post, but it does not automatically know whether every project detail is true. Human review is therefore necessary to verify the facts, remove unsupported claims, and decide whether the final post accurately represents my actual work.

14. What Worked Well

The workflow worked well because each stage had a different purpose. The Draft stage quickly created a starting version, while the Critique stage provided a separate review instead of immediately accepting the first response. The critique was particularly useful for identifying repetition, unnecessary wording, and potential ambiguity. The Revise stage then used that feedback to produce a cleaner final version. Testing the workflow on five different projects also showed that the same process could be reused for different types of technical and creative work.

15. What I Would Change

If I ran the workflow again, I would make the project input more structured by using the same fields for every project, such as purpose, features, technologies, completed work, challenges, and learning. I would also make the Critique stage focus even more strongly on factual accuracy, repetition, and personal voice. Another improvement would be to make the final posts slightly shorter and more suitable for LinkedIn scanning. This would make the workflow more consistent and reduce the amount of human editing required after the revision stage.

16. Final Workflow

Final pipeline:

```
Input
↓
Draft
↓
Critique
↓
Revision
↓
Final Output
↓
Human Review
```

Number of distinct steps: 3

Tool used: Claude Project

Number of real inputs tested: 5

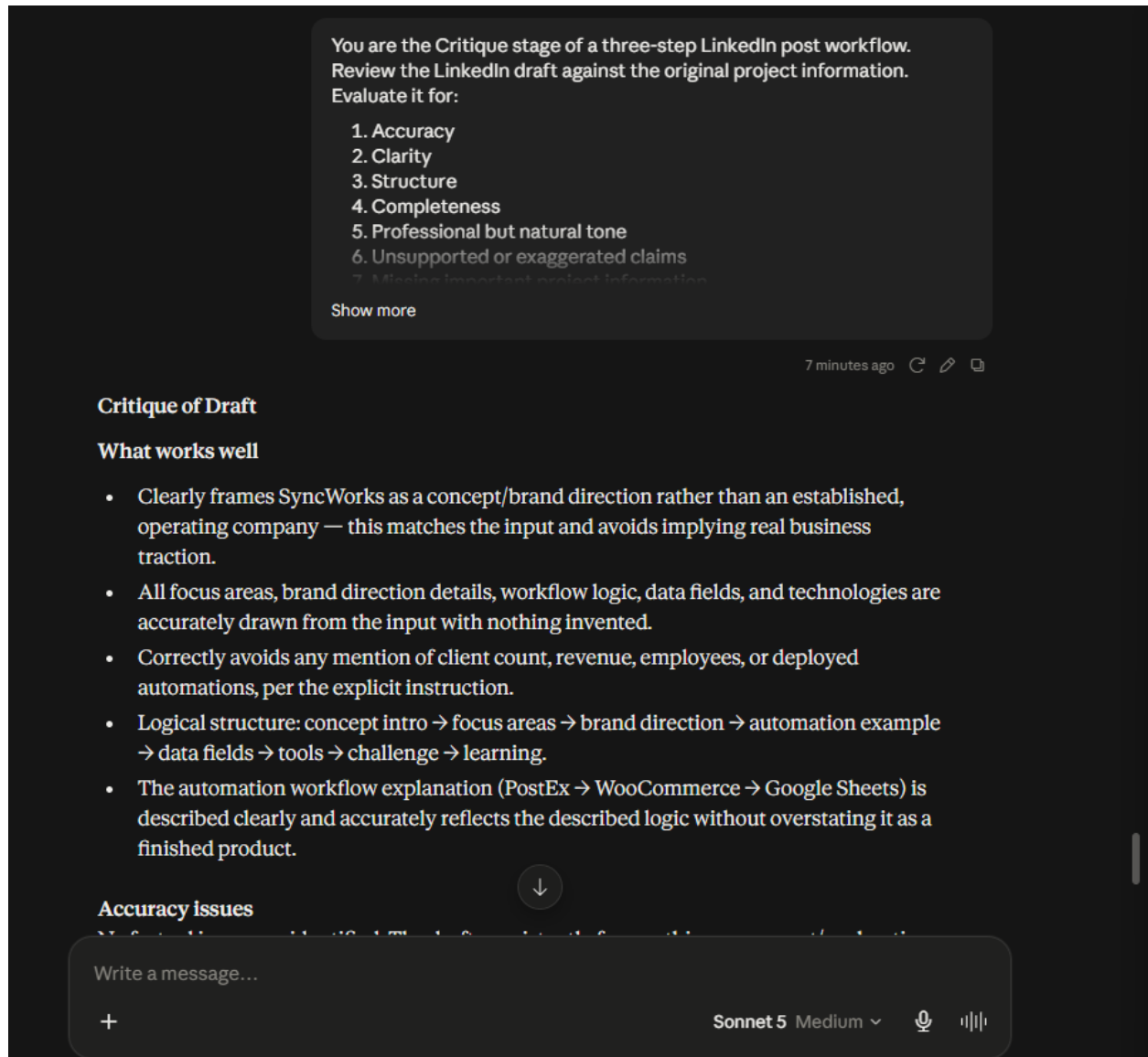
Workflow completed end-to-end: Yes

17. Final Reflection

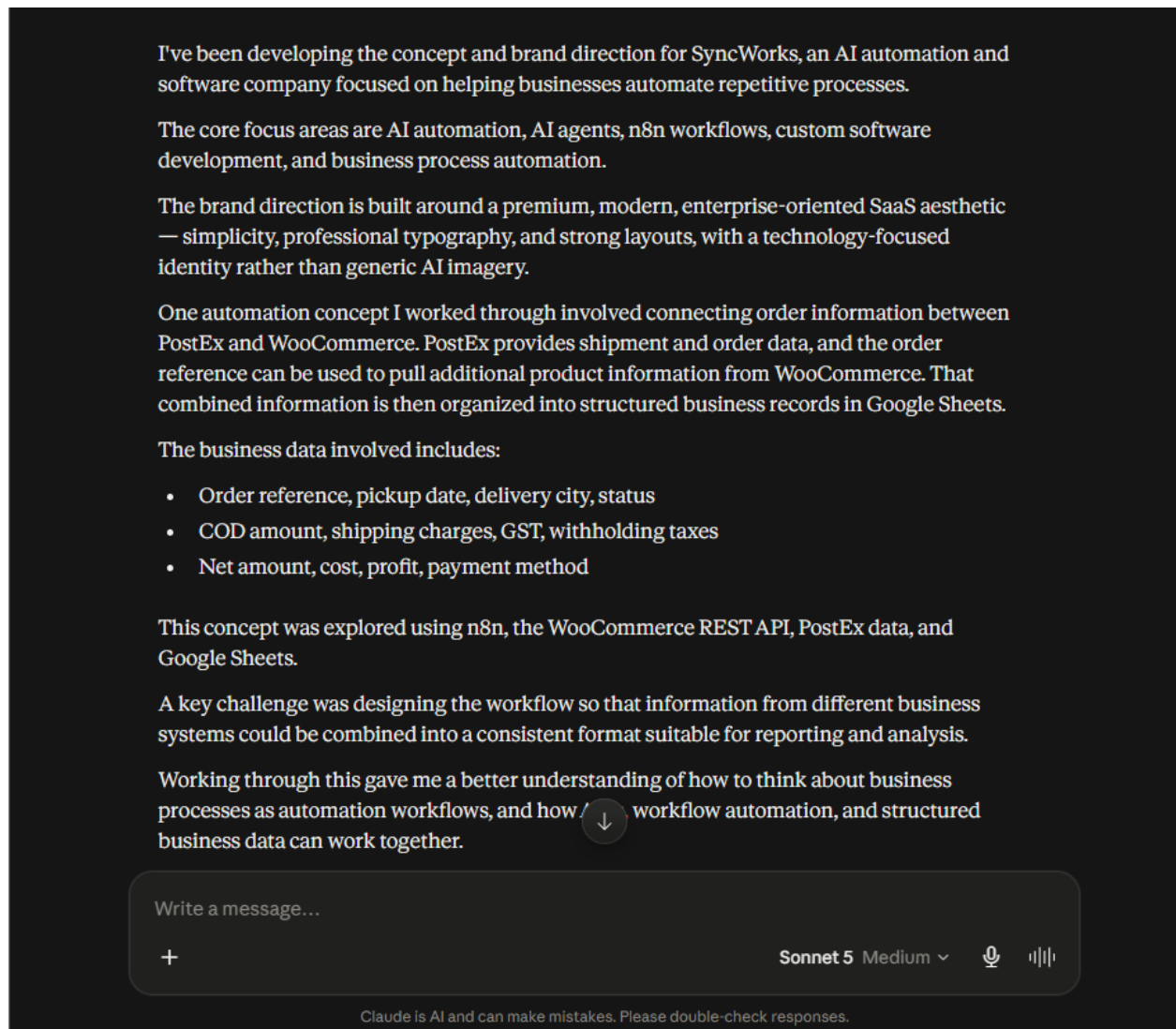
The workflow was useful because it made LinkedIn post creation more structured and repeatable. Instead of relying on one AI-generated response, I separated the process into drafting, critique, and revision. Based on the five-task test, the workflow took about 52 minutes including setup and human checking compared with an estimated 85 minutes for completing the five posts manually, resulting in an estimated saving of 33 minutes. However, the workflow did not remove the need for human judgment. I still needed to verify factual accuracy, project details, claims, and whether the final wording matched my own work. I would continue using this workflow because the main benefit was not only time savings but also having a consistent review process for every post.

18. Evidence

Workflow Running Screenshot



Example Output Screenshot



19. Self-Check

- ☒ The workflow has at least three distinct steps.
- ☒ Each step has a defined input and output.
- ☒ The handoff between steps is clear.
- ☒ The workflow works with a brand-new input.
- ☒ I tested it on five real inputs.
- ☒ I documented the outputs from all five runs.

- ☒ I recorded setup time and actual execution time.
- ☒ I identified where the workflow failed or needed improvement.
- ☒ I identified what a human still needs to check.
- ☒ I included the exact prompts/configuration used.
- ☒ I included screenshots/evidence of the working workflow.
- ☒ The final document reflects what I actually did.